

Nuestra Cafetería

David Priego Puga

3 de diciembre de 2025

Índice

1. Introducción	1
2. Arquitectura de nuestro programa	1
2.1. Clase Cliente	2
2.2. Clase Camarero	2
2.3. Clase Barista	3
2.4. Clase Buffer	3
2.5. Clase CafeController	4
3. Patrón de Diseño	4
4. Complicaciones	5

1. Introducción

En este proyecto pequeño donde seguiremos aprendiendo a trabajar con **hilos** en **java** tenemos que construir una **cafetería** con camareros, clientes y barista, añadiendo además un buffer para poder trabajar con el patrón de diseño **productor-consumidor**. Donde nuestros camareros tendrán que atender a los diferentes clientes que lleguen a la **cafetería**. Los clientes por otro lado cuando llegan a nuestro local tienen un tiempo de espera el cual es aleatorio (Esto está hecho de esta manera para darle un añadido de aleatoriedad y comprobar que los hilos funcionan bien debido a que cada ejecución del programa es completamente aleatoria). Lo único que no es aleatorio es el orden de llegada de los clientes y los camareros, por otro lado tenemos al **barista** el cual se encarga de preparar los cafés de la cafetería hasta el punto en que el **buffer** (Espacio donde se guardan los cafés) hasta que este lleno, en ese momento el barista se irá a "dormir" el camarero empezará a trabajar cogiendo los cafés del buffer hasta el punto que no haya, de tal manera que en el momento que no haya cafés en el buffer el camarero se irá a dormir y el barista nuevamente empezará de nuevo a funcionar.

Por otro lado volviendo a los clientes, estos pueden llegar a la cafetería, ser atendidos dentro de su **tiempo de espera** e irse con el café, por otro lado pueden llegar a la cafetería, ser atendidos pero que el camarero tarde demasiado en servirles entonces se irán enfadados y sin café y por último es que directamente llegue a la cafetería y pase el tiempo y los camareros no lleguen a atender a este cliente de tal manera que no le dará tiempo ni a pedir un café. Esto en resumidas cuentas es el resumen de nuestro programa.

2. Arquitectura de nuestro programa

Nuestro programa se basa en un código realizado en java de tal manera que tendremos de **3 clases diferentes: Camarero, Cliente, Main, Barista y Buffer**. De tal manera que solo con estas clases se consiga la completa gestión de la cafetería.

Además como es un programa sencillo no disponemos de backend ya que usamos una **Lista** de clientes embebida dentro de nuestro **JavaFX**.

2.1. Clase Cliente

En esta clase definimos a cada uno de nuestros clientes. Tenemos dos atributos para cada uno de nuestros **clientes**, cada cliente tiene **nombre** y **tiempo de espera**.

- Nombre: Atributo **String** que indica como se llama el cliente.
- TiempoEspera: Atributo **Int** el cual se calcula mediante un **Math.random** el cual calcula un tiempo aleatorio entre 3 y 4 segundos.

Por otro lado tenemos los diferentes **getters** para cada cliente de tal manera que tenemos:

- getNombre: Método que nos devuelve el nombre del cliente,
- getTiempoEspera: Método que nos devuelve el **TiempoEspera** en un formato **Int**.

Con esto podemos ver que realmente la clase **Cliente** es una clase sencilla que lo único que hace es guardar los datos mínimos para que el programa tenga sentido en relación al cliente.

2.2. Clase Camarero

En esta clase definiremos a cada uno de los posibles camareros que tenga la cafetería pero además de que realmente quien realiza todo el **"trabajo"** es la clase **Camarero** ya que como hemos visto antes, cliente solo manda los datos mínimos imprescindibles.

En caso de los camareros tenemos bastantes más atributos y métodos.

Los atributos de la clase **Camarero** son:

- **String nombre**: De la misma manera que un cliente tiene un nombre en este caso tenemos un nombre para cada uno de los camareros.
- **Cliente[] clientes**: La propia lista de **clientes** se pasa como atributo para la clase camarero de tal manera que así sabe cada camarero cuantos clientes tiene. Podrían ser listas separadas pero para este **proyecto** usaremos la misma lista para todos los camareros.
- **Int indice**: Este número se encargara de marcar el progreso en un **método** posterior llamado **siguienteCliente** de tal manera que el **indice** marca el número en la lista del cliente actual. Por ejemplo el cliente con indice 0 es el cliente en posición 0 del Array por lo tanto el primero.
- **Object lock**: Es un objeto candado que usamos en el método **"siguienteCliente"** de tal manera que se usa como **bloqueo** debido a que los dos camareros interactúan con la misma lista esto hace que cuando un camarero está trabajando con la lista y aumentando el indice, el **lock** prohíbe al resto de camareros entrar en la lista de clientes y modificar el **indice**.
- **VBox contenedorMensajes**: Contenedor de cada camarero donde vamos a imprimir cada uno de los mensajes que se generan en nuestro programa de tal manera que sabemos a que camarero pertenece cada uno de los mensajes de cada cliente.
- **Buffer buffer**: Clase que declaramos como recipiente" donde se guardan los cafés hasta que el camarero los recoja de tal manera que podemos hacer el símil de una **"bandeja"** clásica de una cafetería el cual el barista llena de cafés y en el momento que está llena el camarero la recoge y va repartiendo todos los cafés por cada una de las mesas de la cafetería.

Con esto tenemos todos los atributos que dispone cada uno de los camareros, como podemos ver son un par más que los que dispone la clase cliente pero sobretodo el peso que estos tienen a la hora de ejecutar el programa.

Pero bueno ahora hablaremos de los diferentes métodos que dispone la clase **Camarero**:

- **siguienteCliente**: Método que sirve para moverse a través de la lista de clientes de tal manera que solo un **camarero** puede estar a la vez, esto se hace para evitar que ambos camareros atiendan al mismo cliente, para lograr esto se hace una comprobación de indices, primero de todo, se bloquea el acceso de otro camarero con **synchronized (lock)**, luego se comprueba que

el índice actual sea menor que la longitud de la lista de clientes. Se recoge el cliente en posición = índice, el índice crece en una unidad y se devuelve el cliente. De tal manera que el primer camarero accederá al cliente en posición cero (Es decir al primer cliente) y el segundo camarero cuando entre al método **siguienteCliente** entrará cuando **índice** sea ya uno por lo que accederá al segundo cliente.

- **mostrarMensaje**: Método que se encarga de imprimir en pantalla los diferentes mensajes que se generan a la hora de atender a los clientes, debido a que en **JavaFX** tenemos un contenedor para cada camarero y ahí imprimimos dichos mensajes.
- **prepararCafe**: Este método se encarga de la gestión de los clientes al comprobar si han sido atendidos a tiempo o no, en este caso mostramos diferentes mensajes de información de tal manera que se sepa en que punto del programa estamos y de tal manera que luego, llamamos a un **método de buffer que es consumir café** de tal manera que el café se resta del propio buffer y no sale del camarero.
- Tenemos por último una comprobación de cliente, si cliente es igual a **siguienteCliente** pero es distinto a **null** entonces llamaremos a **prepararCafe** si no se da esta condición implica que la lista de clientes se terminó por lo que mostramos un mensaje donde se indica que el **camarero terminó el turno**.

2.3. Clase Barista

En esta clase definimos a un barista que se encarga de "hacer" los cafés para todos los clientes de la cafetería, esta clase solo tiene comunicación con un buffer de a su vez se comunica directamente con camareros de tal manera que esta clase podríamos decir que solo se comunica con camareros, de manera que hace funcionar el patrón de diseño **productor-consumidor** que explicaremos mas adelante,

En esta clase tenemos los siguientes atributos:

- **private Buffer buffer**: Le pasamos a la clase barista el buffer donde va a guardar los cafés creados.
- **private boolean activo**: La etiqueta que marca si barista está o no creando cafés.
- **private VBox contenedorMensajes**: Contenedor del barista donde vamos a imprimir cada uno de los mensajes que se generan en nuestro programa de tal manera que sabemos cuando el barista está trabajando o "durmiendo".

Con esto tenemos todos los atributos que dispone el barista. Ahora procederemos a ver que **métodos** tiene **Barista**

- **MostrarMensajes**: Método que se encarga de imprimir en pantalla los diferentes mensajes que se generan a la hora de hacer cafés, los cuales se imprimen en el contenedor de mensajes anteriormente declarado.
- **public void run()**: Donde indicamos mientras el barista esté activo donde tenemos un contador para los cafés, además de los mensajes pertinentes a la hora de gestionar como se hacen los cafés. Además de el llamado a la **Clase Buffer** y su **método de producir(café)**.

2.4. Clase Buffer

En esta clase trabajaremos ese contenedor donde guardaremos todos los cafés que se van a estar intercambiando **barista y camareros** para ello creamos este **buffer** el cual como atributos tenemos:

- **private final int capacidad**: Donde indicamos la capacidad máxima de nuestro buffer.
- **private final Queue<String> buffer = new LinkedList<>()**: Una Lista con cola donde se mantiene el orden de los elementos que entran en dicha lista.

Ahora vamos a ver los diferentes métodos que tenemos en nuestra clase **Buffer**:

- **public synchronize void producir(String café):** método donde añadimos café donde tenemos un if en caso de que el `buffer.size` tenga sea igual a la capacidad (es decir que este lleno) pues mandamos un **wait** de tal manera que espera y si no **añadimos** un café de tal manera que lo hacemos y notificamos a todos los hilos para que se conecten con el **buffer** a través de un **notifyAll()**
- **public synchronize void consumir():** Es el método contrario, este método mientras este vacío espera y si no quitamos el café del buffer con un **buffer.poll()** y cuando esto sucede mandamos otro **notifyAll()** al resto de hilos.

2.5. Clase CafeController

Esta clase en **JavaFX** es la que substituye a la clase **Main** en nuestro programa en **Java**. Pero de base lo que tenemos en este archivo es:

- El **VBox** que nos los contenedores para cada uno de nuestro contenedores en este caso contenedores para los camareros y el contenedor para el barista.

Aparte tenemos 3 métodos:

- **iniciarSimulación:** Método donde tenemos definido nuestro array de **clientes**, además creamos las tarjetas para cada uno de los camareros que queramos crear, en este caso tenemos 2 camareros creados y aparte creamos la tarjeta para nuestro barista que esté también debe aparecer en la interfaz.
- **crearTarjeta:** Método que crea el contenedor para la tarjeta de cada uno de los camareros.
 - Contenedor Tarjeta **VBox**
 - Label con el nombre del camarero.
 - Contenedor **VBox** que es el contenedor de los diferentes mensajes.
 - Luego a **Tarjeta** añadimos el label del nombre del camarero y el contenedor de mensajes
 - Por último creamos el camarero con su constructor y iniciamos el **hilo**.
- **crearTarjetaBarista:** Método que crea el contenedor para la tarjeta del barista de la misma manera que creamos la del camarero.
 - Contenedor tarjeta **VBox**
 - Label para el nombre del Barista
 - Contenedor **VBox** que es el contenedor o caja para los mensajes que provienen de barista.
 - Luego añadimos a la **tarjeta** el label del barista y el contenedor de los mensajes

3. Patrón de Diseño

En este programa hemos implementado un patrón de diseño llamado **productor-consumidor**, el cual se basa en la comunicación tal cual de una clase que se encarga de "producir" (productor) y otra clase que se encarga de "consumir" lo producido por el productor (consumidor) de tal manera que se crea un flujo de trabajo entre las dos clases que se comunican de tal manera a través de un **contenedor** donde se guardan los diferentes elementos que se producen y consumen. En este caso se crea un **buffer** donde se guardan y se retiran los cafés de los baristas a los camareros.

Para que se entienda de una manera resumida:

- **Barista** hace cafés - **Productor**.
- **Camarero** sirve cafés - **Consumidor**.
- **Buffer** bandeja donde se guardan cafés - **Intermediario**.

Las reglas básicas de este método son las siguientes:

- El **barista** solo puede guardar cafés si el buffer está vacío.
- El **Camarero** solo puede coger cafés cuando el **buffer** está lleno.
- Esto todo se realiza de una manera ordenada donde los cafés solo pueden entrar o salir y nunca a la vez.

4. Complicaciones

Para crear este programa en **JavaFX** recolectando las clases del ejercicio anterior y modificando para que funcione con el patrón de diseño para ello tuve que planear como gestionar el **productor-consumidor** de tal manera que tuvimos que crear la clase buffer y la clase barista, la clase barista fue sencilla de crear pero la clase **buffer** me costó pillar le el truco, tuve que buscar información, volver a ver el PDF. Además de que tuve que preguntar por internet porque la verdad que en todo este proceso, me ha costado entender el procedimiento, sobretodo de los métodos que funcionan a la inversa a la hora de llamar a los hilos.